FedLight: Federated Reinforcement Learning for Autonomous Multi-Intersection Traffic Signal Control

Yutong Ye[1], Wupan Zhao[1], Tongquan Wei[1], Shiyan Hu[2] and Mingsong Chen[1]
[1]MoE Research Center of HW/SW Co-design, East China Normal University, Shanghai, 200062, China
[2]School of Electronics and Computer Science, University of Southampton, Southampton SO17 1BJ, United Kingdom

2021 58th ACM/IEEE Design Automation Conference (DAC) | 978-1-6654-3274-0/21/$31.00 ©2021 IEEE | DOI: 10.1109/DAC18074.2021.9586175

***Abstract*—Although Reinforcement Learning (RL) has been successfully applied in traffic control, it suffers from the problems of high average vehicle travel time and slow convergence to optimized solutions. This is because, due to the scalability restriction, most existing RL-based methods focus on the optimization of individual intersections while the impact of their cooperation is neglected. Without taking all the correlated intersections as a whole into account, it is difficult to achieve global optimization goals for complex traffic scenarios. To address this issue, this paper proposes a novel federated reinforcement learning approach named FedLight to enable optimal signal control policy generation for multi-intersection traffic scenarios. Inspired by federated learning, our approach supports knowledge sharing among RL agents, whose models are trained using decentralized traffic data at intersections. Based on such model-level collaborations, both the overall convergence rate and control quality can be significantly improved. Comprehensive experimental results demonstrate that compared with the state-of-the-art techniques, our approach can not only achieve better average vehicle travel time for various multi-intersection configurations, but also converge to optimal solutions much faster.**

***Index Terms*—Federated Reinforcement Learning, Neural Network, Advantage Actor-Critic, Traffic Signal Control**

I. Introduction

Along with the prosperity of Artificial Intelligence (AI) and Internet of Things (IoT) technology, Reinforcement Learning (RL) is becoming popular in the design of control models for traffic-oriented autonomous systems, especially for traffic light control [1], [2]. To enable RL-based autonomous signal control for a transportation network, typically each intersection is equipped with an agent that generates actions to signal all the traffic lights of the intersection based on real-time traffic data captured by IoT devices, e.g., traffic cameras, roadside sensors. For each traffic intersection, the set of actions applied on all the involved traffic lights form a control phase indicating the rights-of-way signaled to vehicles. Based on the optimized control phases made by intelligent agents, the average travel time of vehicles in the network can be effectively reduced. Meanwhile, all the collected traffic data is used for the RL training on-the-fly to incrementally construct more accurate control models.

Although RL is promising in deriving accurate traffic control policies, its usage is greatly restricted by the issue of scalability. This is because the complexity of RL training for traffic control relies on two key factors: i) environment simulation model for traffic network, and ii) the number of possible actions for each state. When more intersections are involved in a traffic network, due to the combination of intersection actions, the exploration space of RL training will increase exponentially. Therefore, instead of applying RL on the whole traffic network, most of the existing RL-based methods focus on the signal control optimization of individual intersections. This will inevitably deteriorate the control quality, since the signal actions made by intersection agents are correlated. Without taking the cooperation of agents into account, the agents may easily get stuck at exploring local optimal solutions rather than figuring out a global optimal one, thus resulting in inferior control effects. Worse still, due to the lack of agent cooperation, in many practical scenarios RL-based methods converge very slowly or even cannot converge. Therefore, *how to enable the cooperation among intersection agents to quickly form an optimal control solution for the whole traffic network* is becoming a major challenge in the design of RL-based traffic signal control.

To address the above challenge, we present a novel Federated Reinforcement Learning (FRL) approach named FedLight that can quickly derive optimal traffic signal control policies for multi-intersection traffic scenarios. Rather than applying one single RL agent on all the intersections, FedLight adopts a distributed way to autonomously control the intersections. In FedLight, each intersection is controlled by an agent based on the Advantage Actor-Critic (A2C) method [3]. The agent consists of a strategic neural network as the Actor that chooses actions for the agent, and a value neural network as the Critic that evaluates actions chosen by the agent. Inspired by the concept of Federated Learning (FL) [4] that supports central model training on decentralized device data, our approach enables the knowledge sharing among agents based on updated gradient and parameter information, thus both the overall RL training speed and the control performance can be improved. This paper makes the following three major contributions:

- We propose a novel cloud-based FRL framework that can effectively coordinate the training processes of all the agents, and form a centralized control model for the optimization of multi-intersection traffic scenarios.
- To improve the convergence rate of training processes and the control quality of A2C agents, we introduce an FL-like gradient aggregation and parameter dispatching mechanism to enable knowledge sharing among agents.
- We implement our proposed FedLight method on top of the simulator Cityflow [5]. Experimental results on various multi-intersection traffic datasets show the effectiveness of our approach.

The rest of this paper is organized as follows. After the introduction to related works in Section II, Section III gives the

This work was supported by the National Key Research and Development Program of China 2018YFB2101300, and Natural Science Foundation of China 61872147. Mingsong Chen is the corresponding author (mschen@sei.ecnu.edu.cn).

Authorized licensed use limited to: Netaji Subhas University of Technology New Delhi. Downloaded on April 29,2026 at 09:45:44 UTC from IEEE Xplore. Restrictions apply.

details of our proposed FedLight approach. Section IV presents the experimental results. Section V concludes the paper.

II. Related Work

To solve traffic congestion problems in urban transportation, various traffic signal control methods have been proposed. Especially in recent years, we have witnessed an increasing trend of RL-based optimization for traffic signal control. For example, through exploring the intuitive principle of phase competition in traffic signal control, Zheng et al. [6] proposed an RL-based method, called FRAP, which can well handle road structures and traffic conditions. Inspired by the max pressure method in the transportation field, Wei et al. [7] developed an RL method named PressLight based on the pressure difference to maximize the throughput of the traffic network. In [8], Chen et al. designed pressure based RL agents to achieve signal coordination in regions. They combined both the FRAP framework and PressLight to expand the scale of signal light control to the city level. Although all these methods are promising in achieving high control quality, most of them adopt the independent Q-Learning framework, where each intersection is controlled by one Deep Q-network (DQN) agent.

By taking the cooperation of intersections into account, the overall control performance can be improved. For example, Chang et al. [9] proposed an effective adaptive back-pressure traffic control algorithm that considers the utilization of all the correlated intersections. However, this work is not based on RL, and the optimization goal is stability. In [10], Wei et al. presented an RL method called CoLight that uses a graph attentional network to incorporate the influences of neighboring intersections for better signal control. Although the neighboring intersection information is useful for locally correlated agents, CoLight cannot form a global observation for all the intersections and support the sharing of locally learned knowledge with all the other intersections within the same traffic network.

As a promising collaborative learning framework, federated reinforcement learning [11] has been proposed to federatively build control models of high-quality for agents. It has been successfully applied in the robot domain for multi-robot collaborative control [12]. Unlike the knowledge fusion work in [12] that uses prior knowledge and quickly adapts to new environment, in this paper we target to share the knowledge among intersection agents to quickly form a solution to achieve optimal average travel time for vehicles. To the best of our knowledge, our work is the first attempt that combines the advantages of both reinforcement learning and federated learning to optimize the signal control of multi-intersection traffic scenarios.

III. Our FedLight Approach

To enable knowledge sharing using federated learning, we implement FedLight based on a cloud-edge architecture, where A2C-based agents are hosted by edge nodes and the cloud server can collect/dispatch gradient/parameter information of agent models from these edge nodes. Figure 1 details the major components and workflow of FedLight. In our approach, we deploy one edge node at each intersection to monitor the traffic information using connected IoT devices (e.g., cameras), make the control decision and actions on traffic lights, and train the A2C models. For one edge node, once capturing new traffic data by connected cameras, the A2C agent will choose one best action to control traffic lights based on its current state. Meanwhile, the state and action information will be saved to the trajectory memory for future learning. The cloud server has four buffers: i) *Actor gradient/parameter buffers* that collect Actor gradient/parameter information from all the agents; and ii) *Critic gradient/parameter buffers* that gather Critic gradient/parameter information from all the agents.

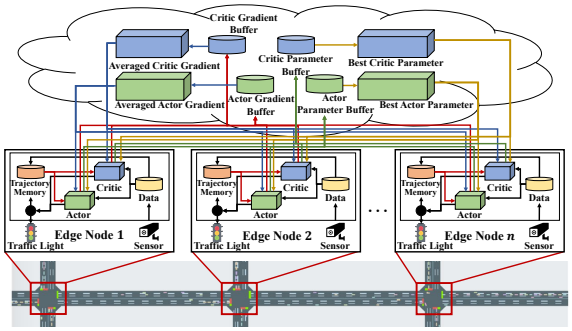


Fig. 1. The workflow of FedLight

In Figure 1, we use purple, red and blue arrow lines to show the workflow of our FL-like gradient aggregation and dispatching mechanism. We use the purple arrow lines to denote the local training of Critic models and Actor models from the trajectory memories, and use red arrow lines and blue arrow lines to denote the collection and dispatching of gradients by the cloud server, respectively. Note that unlike traditional FL which dispatches aggregated models to edge devices, our approach only calculates the average Actor/Critic gradients and dispatches them directly to the edge devices. Besides gradient information, at the end of one episode (i.e., one round of complete simulation), each agent will send its reward together with the parameters of its Actor model and Critic model to the cloud server as shown by green arrow lines. Then the cloud server needs to figure out the Actor/Critic parameters with the best reward and dispatch them to all the agents to replace their Actor/Critic models shown by yellow arrow lines. Based on the above two kinds of knowledge sharing mechanisms (i.e., gradient sharing and parameter sharing), FedLight can improve the overall performance of traffic signal control. The following subsections will give the details of our approach.

A. Intersection Modeling

An intersection consists of an infinite set of arrival lanes L_{arrive} and an infinite set of departure lanes L_{depart}. One road of the intersection can be represented by a 2-tuple (l_a, l_d), where $l_a \in L_{arrive}$ and $l_d \in L_{depart}$. Figure 2 shows an intersection example with 12 arrival lanes (i.e., $\{l_1, l_2, ..., l_{12}\}$) and 12 departure lanes (i.e., $\{l'_1, l'_2, ..., l'_{12}\}$). Note that at the end of each arrival lane, there is a direction mark on the ground, indicating the direction of the following departure lane. One road (l_a, l_d) is a directed road if l_d is the departure road indicated by the direction mark on the ground of l_a. For example, (l_2, l'_2) and (l_7, l'_7) are two directed roads. According to common sense, the lanes to turn right are not restricted by traffic signals. Therefore, this intersection only has eight traffic lights. Typically, an intersection has a set of eight feasible

Authorized licensed use limited to: Netaji Subhas University of Technology New Delhi. Downloaded on April 29,2026 at 09:45:44 UTC from IEEE Xplore. Restrictions apply.

control phases (i.e., $\{p_1, ..., p_8\}$) as shown at the right side of Figure 2 indicating the rights-of-way signaled to vehicles by traffic lights. The intersection example at the left side shows a scenario with control phase 4 enabled, where the vehicles on lane l_3 can turn left to enter lane l'_3, and the vehicles on lane l_7 can turn left to lane l'_7. Note that when the number of lanes increases at an intersection, the number of control phases will not change.

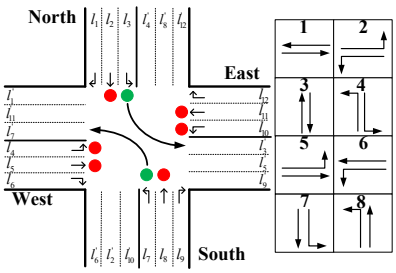
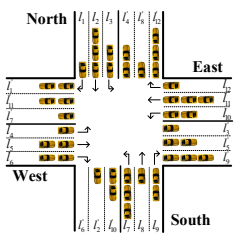


Fig. 2. An intersection example

B. Agent Design

Inspired by max pressure control theory [13], we model the agent rewards based on the pressure of roads and intersections. Let $V(l_a)$ and $V(l_d)$ denote the number of vehicles on the arrival lane l_a and departure lane l_d, respectively. The pressure of a road (l_a, l_d) can be calculated by $V(l_a) - V(l_d)$, and the pressure of an intersection can be calculated by $\Sigma_{l_a \in L_{arrive}} V(l_a) - \Sigma_{l_d \in L_{depart}} V(l_d)$. Figure 3 shows an example of pressure calculation. From this figure, we can find that road pressure and intersection pressure metrics can approximately reflect the vehicle queue lengths of roads and intersections. For the same traffic scenario, if the intersection pressure can be controlled at a low level, the throughput of vehicles crossing this intersection will be maximized.



Road	Pressure	Road	Pressure
(l_1, l_7)	1−2=−1	(l_7, l_7)	3−1=2
(l_2, l_2)	3−1=2	(l_8, l_8)	1−1=0
(l_3, l_3)	2−1=1	(l_9, l_9)	2−3=−1
(l_4, l_4)	1−2=−1	(l_{10}, l_{10})	2−2=0
(l_5, l_5)	2−2=0	(l_{11}, l_{11})	3−2=1
(l_6, l_6)	2−0=2	(l_{12}, l_{12})	1−3=−2

Pressure of intersection
= |# of arrival vehicles − # of depart vehicles|
= |1+3+2+1+2+2+3+1+2+2+3+1 − 2 − 1 − 1 − 2 − 2 − 0 − 1 − 1 − 3 − 2 − 2 − 3|
= 3

Fig. 3. An example of calculating road pressure and intersection pressure

Based on the definition of intersection pressure, we design the A2C agent for FedLight using the following settings.

- **Observation**: Each agent should observe the pressures of all the directed roads of the intersection and the current phase p.
- **Action**: Based on the observation results, the agent needs to select one best control phase to maximize the throughput of the intersection.
- **Reward**: When an action finishes, the environment returns a reward as $r = -P$, where P is the intersection pressure.

In our approach, each A2C agent has two neural networks, i.e., Actor model (denoted by π) and Critic model (denoted by Q). We use $\theta(\pi)$ and $\theta(Q)$ to specify the parameters of π and Q, respectively. The Critic model is used to evaluate the actions selected by the Actor model. During the training, the parameters of Critic model are updated based on the Time Difference Error (TD-error) loss function shown as follows:

$$L(Q) = \frac{1}{2}(R(s,a) + \gamma Q(s',a') - Q(s,a \mid Q))^2 \qquad (1)$$

where $R(s,a)$ is the reward obtained by executing an action a in the state s, s' is the next environment state after the agent executes an action a, the next action a' is sampled by $a' \sim \pi(a \mid s')$, and $\gamma \in (0,1]$ is the discount factor of future rewards. In the formula, $R(s,a)+\gamma Q(s',a')$ is the TD-target. For the Actor model, its gradient is updated based on the following formula:

$$\nabla L(\pi) = E_\pi[A_\pi(s,a)\nabla\pi log\pi(a \mid s)]. \qquad (2)$$

Here, $A_\pi(s,a)$ is the advantage function defined as follows:

$$A_\pi(s,a) = Q_\pi(s,a) - V_\pi(s) \qquad (3)$$

where $Q_\pi(s,a)$ is the value function and $V_\pi(s)$ is the state value function.

C. Implementation of Federated Reinforcement Learning

In this section, we mainly introduce the implementation of the training process for FedLight. Algorithm 1 presents the implementation of the training process for one edge device containing one A2C agent.

At the beginning of each episode, line 1 flushes the trajectory memory and initializes the accumulated reward. Lines 2-20 give the details of one episode step during the reinforcement learning. In steps 2-4, the edge device interacts with the environment and obtains state s, action a, reward $r = R(s,a)$ and the next state s'. Line 5 updates the accumulated reward in one episode. Lines 6-8 calculate the TD-target and the advantage function, and save them in the trajectory memory. Note that in our approach the trajectory memory has five slots. When the trajectory memory is full, a batch of samples is collected. Then in lines 9-14, the Actor model and Critic model calculate the gradients based on batch samples according to Equations (1) and (2). Line 15 averages these gradients, and line 16 uses the average values to update both Actor model and Critic model. In line 17, the edge node sends the averaged gradients to the cloud server for knowledge sharing. Line 18 waits for the aggregated gradients from the cloud server, and line 19 updates the local Actor model and Critic model using the aggregated gradients. Line 20 resets the trajectory memory for the training of the next episode step. At the end of one episode, the Actor model and Critic model send their parameters to the cloud server for the reward comparison in line 21. Once the edge node receives the best parameters from the cloud server in line 22, line 23 will replace the two models using the parameters.

Algorithm 2 presents the implementation of the training process for the cloud server. Lines 1-6 deal with the gradient information collected from all the edge nodes in an FL-like manner. Line 1 initializes all kinds of buffers. Line 2 parses the messages uploaded by edge nodes, and saves the Actor gradients and Critic gradients to the Actor gradient buffer and Critic gradient buffer, respectively in lines 3-4. Line 5 averages the gradients in both the buffers, and sends them to all the edge nodes in line 6. When all the steps in an episode finish, lines 7-10 try to figure out the best Actor model and Critic model trained so far. Line 7 parses the messages from edge nodes, and saves the Actor parameters and Critic parameters in their

Authorized licensed use limited to: Netaji Subhas University of Technology New Delhi. Downloaded on April 29,2026 at 09:45:44 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: Training Procedure for Edge Node

Inputs: i) R, # of episodes; ii) S, # of episode steps; iii) M, trajectory memory; iv) γ, discount factor; v) η, learning rate; vi) σ, accumulated reward in one episode;
for $i \leftarrow$ *1 to* R **do**
  1. $M \leftarrow \{\}$, $\sigma \leftarrow 0$
  for $j \leftarrow$ *1 to* S **do**
    2. $s \leftarrow Observation()$
    3. $a \leftarrow \pi(s)$
    4. $s', r \leftarrow EnvNextStep(a)$
    5. $\sigma \leftarrow \sigma + r$
    6. $TD_{target} \leftarrow r + \gamma Q(s', a')$
    7. $A_\pi \leftarrow Q_\pi(s, a) - V_\pi(s)$
    8. $M \leftarrow M \cup \{< s, a, TD_{target}, A_\pi >\}$
    if M *is full* **then**
      9. $L_A \leftarrow \{\}, L_C \leftarrow \{\}$
      10. $B \leftarrow GetBatch(M)$
      for $sample \in B$ **do**
        11. $\nabla L(\pi) \leftarrow E_\pi[A_\pi(s,a)\nabla\pi log\pi(a \mid s)]$
        12. $L_A \leftarrow L_A \cup \{\nabla L(\pi)\}$
        13. $\nabla L(Q) \leftarrow \nabla(\frac{1}{2}(R(s,a)+\gamma Q(s',a')-Q(s,a \mid Q))^2)$
        14. $L_C \leftarrow L_C \cup \{\nabla L(Q)\}$
      end
      15. $[\nabla_A, \nabla_C] = [AVG(L_A), AVG(L_C)]$
      16. $[\pi, Q] \leftarrow [\pi - \eta\nabla_A, Q - \eta\nabla_C]$
      17. $SendToCloud1(\nabla_A, \nabla_C)$
      18. $\overline{\nabla}_A, \overline{\nabla}_C \leftarrow ReceiveFromCloud1()$
      19. $[\pi, Q] \leftarrow [\pi - \eta\overline{\nabla}_A, Q - \eta\overline{\nabla}_C]$
      20. $M \leftarrow \{\}$
    end
  end
  21. $SendToCloud2(\theta(\pi), \theta(Q), \sigma)$
  22. $\theta_{best}(\pi), \theta_{best}(Q) \leftarrow ReceiveFromCloud2()$
  23. $[\theta(\pi), \theta(Q)] \leftarrow [\theta_{best}(\pi), \theta_{best}(Q)]$
end

Algorithm 2: Training Procedure for Cloud Server

Inputs: i) N, # of edge devices ; ii) R, # of episodes; iii) S, # of episode steps; iv) AP, buffer of Actor parameters; v) CP, buffer of Critic parameters; vi) AG, buffer of Actor gradients; vii) CG, buffer of Critic gradients; viii) RW, buffer of rewards;
for $t \leftarrow$ *1 to* R **do**
  for $j \leftarrow$ *1 to* S **do**
    1. $AP \leftarrow CP \leftarrow AG \leftarrow CG \leftarrow \{\}$
    for $e \leftarrow$ *1 to* N **do**
      2. $(\nabla_{A_e}, \nabla_{C_e}) \leftarrow ReceiveFromEgde1(e)$
      3. $AG \leftarrow AG \cup \{\nabla_{A_e}\}$
      4. $CG \leftarrow CG \cup \{\nabla_{C_e}\}$
    end
    5. $[\overline{\nabla}_A, \overline{\nabla}_C] \leftarrow [AVG(AP), AVG(CP)]$
    6. $SendToAllEdge1(\overline{\nabla}_A, \overline{\nabla}_C)$
  end
  for $e \leftarrow$ *1 to* N **do**
    7. $(\theta_{A_e}, \theta_{C_e}, \sigma_e) \leftarrow ReceiveFromEgde2(e)$
    8. $AP \leftarrow AP \cup \{\theta_{A_e}\}$
    9. $CP \leftarrow CP \cup \{\theta_{C_e}\}$
    10. $RW \leftarrow RW \cup \{\sigma_e\}$
  end
  11. $[\theta_{A_{best}}, \theta_{C_{best}}] \leftarrow ChooseBest(AP, CP, RW)$
  12. $SendtoAllEdge2(\theta_{A_{best}}, \theta_{C_{best}})$
end

corresponding buffers. Line 10 buffers the reward, which is not explicitly shown in Figure 1. When all the Actor and Critic parameters are collected, line 11 will choose the best Actor and Critic parameters of all the agents based on their reward information, and line 12 will send them to all the edge nodes.

IV. Experimental Results

To evaluate the effectiveness of our proposed FedLight method, we conducted experiments on a Linux workstation with 2.5GHz Intel Xeon CPU, 32GB memory, and NVIDIA Tesla V100 (16G) GPU. By using Python, we implemented our framework on top of the simulator Cityflow [5], which is used to construct environment for traffic networks. Similar to the work in [7], [8], [10], we set the phase duration of each signal light to 10 seconds. During the simulation, there are four incoming approaches and four outgoing approaches in each intersection, where each approach has three lanes of left-turn, through and right-turn. Vehicles can enter/leave the network using the rim edges. We collected all the four multi-intersection benchmark datasets from [15] as the inputs of Cityflow to simulate different traffic scenarios with a time span of 3600 seconds, respectively. The dataset details are as follows:

- **Synthetic datasets**: The three datasets (i.e., Syn_1, Syn_2, Syn_3) represent the traffic scenarios with 2×2, 3×3, and 4×4 intersections, respectively. Following the popular simulation setup in the literature, in each entering edge the number of generated vehicles is assumed to follow a Gaussian distribution, and the turning ratios are set to 10% for turning left, 60% for going straight and 30% for turning right.
- **Real-word datasets**: The dataset Real_1 was collected by roadside surveillance cameras, which reflects some real-world traffic scenarios (Dongfeng sub-district with 3×4 intersections) of Jinan city in China.

In our approach, each intersection is equipped with one agent involving two deep neural networks, i.e., Actor model and Critic model, which have the same structure of hidden layers (two layers with 64 and 32 nodes, receptively) and adopt the same Adam optimizer to update parameters. We set the batch size of local training for Critic models and Actor models to 5. The learning rates η for Actor model and Critic model are 0.0001 and 0.0002, respectively. The discount factor γ is 0.95.

A. Results of Control Quality

To check the control quality of FedLight in terms of average travel time, we compared FedLight with the following six methods: i) *Fixed Time* method [16] with the preset fixed phase time and control schedule; ii) *MaxPressure* [13] based on the max pressure control; iii) SOTL [14] that is an adaptive control based on the number of waiting vehicles; iv) GRL [1] that is an RL-based method with DQN; v) *CoLight* [10] that is an

Authorized licensed use limited to: Netaji Subhas University of Technology New Delhi. Downloaded on April 29,2026 at 09:45:44 UTC from IEEE Xplore. Restrictions apply.

RL-based method considering the neighboring intersection information; and vi) *A2C* that is an RL-based method controlling one intersection with an individual A2C agent.

TABLE I
AVERAGE TRAVEL TIME OF VEHICLES USING DIFFERENT CONTROL

Type	Model	Average Travel Time (s)			
		Syn_1	**Syn_2**	**Syn_3**	**Real_1**
Non-RL	Fixed Time [16]	601.67	746.01	893.02	869.85
	MaxPressure [13]	477.71	631.75	689.68	361.33
	SOTL [14]	482.06	465.47	519.63	453.18
RL	GRL [1]	251.32	422.12	534.23	551.32
	CoLight [10]	312.29	328.70	397.07	327.65
	A2C	140.35	170.92	300.2	511.04
	FedLight	**129.36**	**167.43**	**213.16**	**283.14**

Table I shows experimental results for different control methods. For the RL methods (i.e., GRL, CoLight, A2C, FedLight), we trained their models using 200 episodes. From this table, we can find that most RL methods have much shorter average travel time than the non-RL methods, and FedgLight achieves the best performance among all the seven control methods for the four datasets. Since the CoLight and A2C use different neural networks to generate actions, the results here do not reflect the advantage of CoLight, which considers the cooperation between adjacent intersections. When the traffic scenarios get more complicated, our FedLight outperforms A2C more significantly. For example, A2C and FedLight have similar results for Syn_2 with 3×3 intersections, while FedLight outperforms A2C by 28.99% for Syn_3 with 4×4 intersections. For Real_1, A2C cannot converge to a satisfactory result, while FedLight can converge quickly and finally outperforms the best case using CoLight by 14.59%. Since FedLight adopts the same RL model as the A2C, the main reason of the improvement is mainly due to the cooperation among intersections.

B. Results of Training Convergence Rate

Figure 4 shows the training processes for A2C and FedLight using different datasets. From this figure, we can find that more episodes will lead to lower average travel time, and FedLight outperforms A2C from the perspectives of both average travel time and the convergence rate.

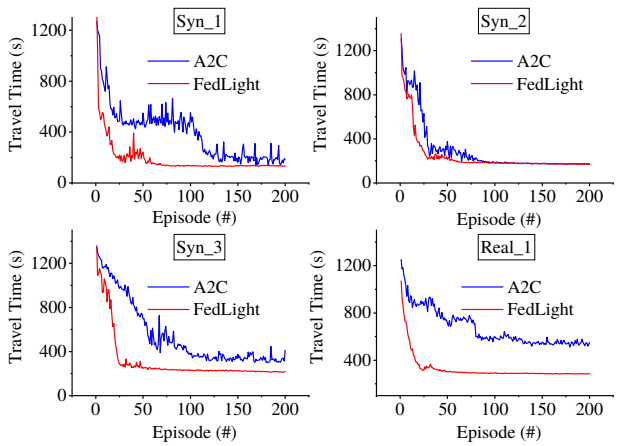


Fig. 4. The training processes of FedLight and A2C.

Table II shows the start episode information for the convergence processes shown in Figure 4. Due to the intersection cooperation, we can find that FedLight has a much earlier convergence time than A2C. For example, for Syn_2 FedLight needs 67 episodes for convergence while A2C needs 101 episodes. Moreover, we can find that for Syn_1, Syn_3 and Real_1, A2C still fluctuates notably around episode 200. However, FedLight converges quickly in these cases. As an example for Real_1, to achieve an average travel time of 511.04 seconds, A2C requires 200 episodes. However, FedLight starts to converge at the 49th episode, and its average travel time is 283.14 seconds.

TABLE II
CONVERGENCE INFORMATION OF FEDLIGHT AND A2C

Model	Start Episode # of Converge			
	Syn_1	**Syn_2**	**Syn_3**	**Real_1**
A2C	–	101	–	–
FedLight	**75**	**67**	**74**	**49**

Unlike Figure 4 that shows the overall training processes for whole traffic networks, Figures 5-8 present the learning process of each A2C or FedLight agent in the traffic network from the perspective of its reward. Since A2C agents within the same traffic network are independent of each other, from these figures we can find that without the knowledge sharing, the A2C agent rewards fluctuate more drastically than the ones of FedLight during the early episodes.

Similar to the convergence trends shown in Figure 4, A2C converges slowly for dataset Syn_2. For datasets Syn_1, Syn_3 and Real_1 shown in Figures 5, 7 and 8, respectively, we can find that some agents still fluctuate drastically (e.g., agent 4 in Figure 5), which clearly explains the reason why the A2C cannot converge for datasets Syn_1, Syn_3 and Real_1 shown in Figure 4.

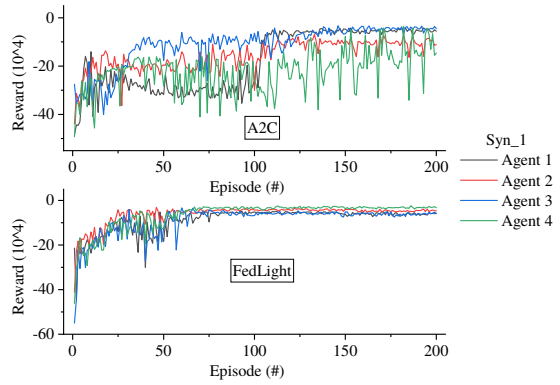


Fig. 5. The learning processes of FedLight and A2C agents on Syn_1

Oppositely, from Figures 5-8 we can observe that FedLight has a much better convergence performance than A2C. This is because by sharing the knowledge learned from their local intersection control with other agents, agents are becoming more and more intelligent, thus making the decisions more accurately with higher rewards. Consequently, FedLight agents can quickly converge to their best statuses for different datasets. As an example in Figure 5, agent 4 of FedLight converges more quickly than its A2C counterpart under the help from other agents via knowledge sharing.

Authorized licensed use limited to: Netaji Subhas University of Technology New Delhi. Downloaded on April 29,2026 at 09:45:44 UTC from IEEE Xplore. Restrictions apply.

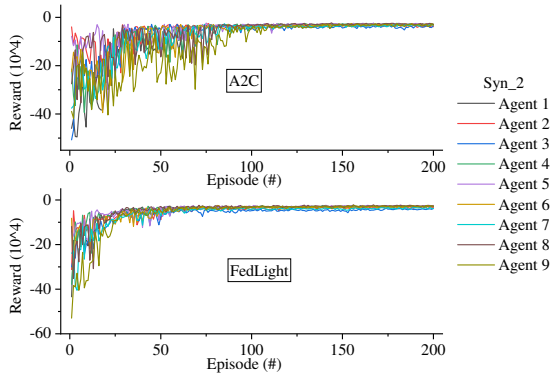


Fig. 6. The learning processes of FedLight and A2C agents on Syn_2

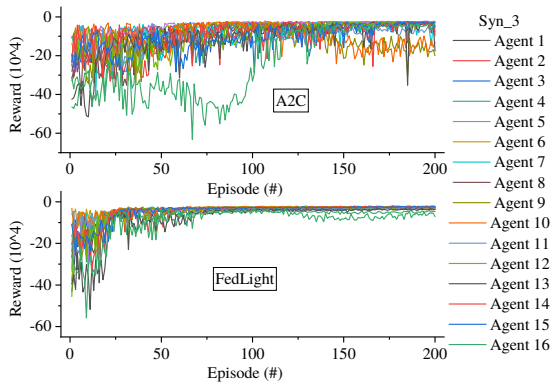


Fig. 7. The learning processes of FedLight and A2C agents on Syn_3

C. Discussion on Communication Overhead

In our approach, each agent is equipped with two neural network models, i.e., Actor and Critic. The Actor model has four layers with 13, 64, 32, and 8 neurons on each layer, respectively. The Critic model has four layers with 13, 64, 32, 1 neurons on each layer, respectively. Therefore, the total number of parameters for Actor is $((13+1)\times 64+(64+1)\times 32+(32+1)\times 8)=3240$, and the total number of parameters for Critic is $((13+1)\times 64+(64+1)\times 32+(32+1)\times 1)=3009$. Since each neuron is encoded by a variable of *float64* type that needs 8 bytes, the size of the gradient sent from an agent to cloud is $(3240+3009)\times 8B = 48.82KB$. In the experiment, we set the phase duration of traffic lights to 10 seconds. It means that in every 10 seconds the agent of an intersection needs to interact with its traffic environment and sample the intersection pressure as a reward. Once the agent collects a batch of five new samples, it will use these samples to update its Actor and Critic models. In other words, in every 50 seconds, each edge node needs to send and receive 48.82KB gradients, respectively. This communication overhead is tolerable for most IoT devices, since in each hour one IoT device only has a communication overhead of $48.82KB \times 2 \times (3600 \div 50 + 1) = 6.96MB$.

V. Conclusion

Although reinforcement learning techniques have been increasingly investigated in traffic signal control, they still suffer from the scalability restriction, resulting in high convergence time and low control quality. To address this problem, this paper proposed an efficient federated reinforcement learning method that can quickly form a globally optimized RL-based agent that can be deployed at all the intersections to improve the overall control performance of traffic networks. Unlike existing methods which focus on the optimization of RL models of independent intersection agents, our approach enables the sharing of the best knowledge among RL agents. In this way, the RL agents deployed at intersections can quickly learn from each other and control the vehicles more intelligently. Experimental results on various complex multi-intersection scenarios show that our approach can not only achieve better control quality in terms of average vehicle travel time, but also can quickly converge to a global optimal solution.

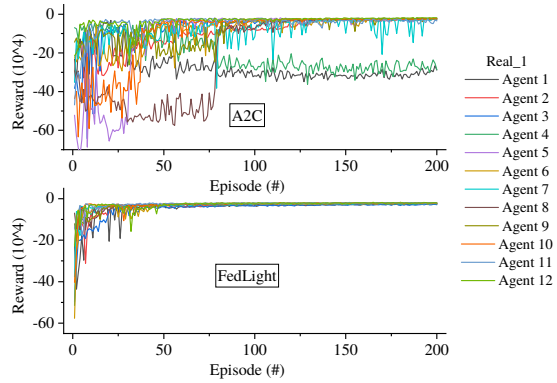


Fig. 8. The learning processes of FedLight and A2C agents on Real_1

References

[1] E. Pol and F. A. Oliehoek, "Coordinated deep reinforcement learners for traffic light control," in *Workshop on Learning, Inference and Control of Multi-Agent Systems*, pp. 1-8, 2016.

[2] H. Wei, et al., "Intellilight: A reinforcement learning approach for intelligent traffic light control," in *Proc. of SIGKDD*, pp. 2496–2505, 2018.

[3] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Machine Learning*, vol. 8, no. 3, pp. 229–256, 1992.

[4] H. B. McMahan, et al., "Communication-efficient learning of deep networks from decentralized data," in *Proc. of AISTATS*, pp. 1273–1282, 2017.

[5] H. Zhang, et al., "Cityflow: A multi-agent reinforcement learning environment for large scale city traffic scenario," in *Proc. of WWW*, pp. 3620–3624, 2019.

[6] G. Zheng, et al., "Learning phase competition for traffic signal control," in *Proc. of CIKM*, pp. 1963–1972, 2019.

[7] H. Wei,et al., "Presslight: Learning max pressure control to coordinate traffic signals in arterial network," in *Proc. of SIGKDD*, pp. 1290–1298, 2019.

[8] C. Chen, et al., "Toward a thousand lights: Decentralized deep reinforcement learning for large-scale traffic signal control," in *Proc. of AAAI*, pp. 3414–3421, 2020.

[9] W. Chang, et al., "CPS-oriented modeling and control of traffic signals using adaptive back pressure," in *Proc. of DATE*, pp. 1686–1691, 2020.

[10] H. Wei, et al., "Colight: Learning network-level cooperation for traffic signal control," in *Proc. of CIKM*, pp. 1913–1922, 2019.

[11] H. H. Zhuo, et al., "Federated reinforcement learning," *arXiv preprint arXiv:1901.08277*, 2019.

[12] B. Liu, L. Wang, and M. Liu, "Lifelong federated reinforcement learning: a learning architecture for navigation in cloud robotic systems," *IEEE Robotics and Automation Letters*, vol. 4, no. 4, pp. 4555–4562, 2019.

[13] P. Varaiya, "The max-pressure controller for arbitrary networks of signalized intersections," in *Advances in Dynamic Network Modeling in Complex Transportation Systems*, pp. 27–66, 2013.

[14] S. Cools, C. Gershenson, and B. D'Hooghe, "Self-organizing traffic lights: A realistic simulation," in *Advances in Applied Self-organizing Systems*, Springer, pp. 41–50, 2013.

[15] Benchmark Dataset, https://traffic-signal-control.github.io/dataset.html.

[16] P. Koonce and L. Rodegerdts, "Traffic signal timing manual." United States. Federal Highway Administration, Tech. Rep., 2008.

Authorized licensed use limited to: Netaji Subhas University of Technology New Delhi. Downloaded on April 29,2026 at 09:45:44 UTC from IEEE Xplore. Restrictions apply.